

Working with AI Coding Assistants

A short practical guide for developers adopting AI pair programming

AI coding assistants have moved from novelty to daily tool in a remarkably short time. A developer who once searched documentation for twenty minutes can now describe a problem in a sentence and receive working code in seconds. That speed is genuinely useful, but it also changes what the job requires. The skill that matters most is no longer recalling syntax; it is knowing how to describe intent precisely and how to judge whether the answer you got back is correct.

Describe the outcome, not the keystrokes

The most common mistake newcomers make is treating an assistant like an autocomplete engine. They ask for a single line, accept it, ask for the next, and end up steering the model one token at a time. This is slower than writing the code by hand and produces work with no coherent shape.

A better approach is to describe the outcome you want and the constraints that matter. State what the code must do, what it must not do, which library versions are in play, and how you will know it worked. An assistant given that context can plan several steps ahead, and the result usually needs far less correction.

Give the assistant the same context you would give a colleague

A new teammate who joins your project reads the README, skims the folder structure, and asks what the conventions are. An assistant benefits from exactly the same material. Point it at the files that matter. Tell it which patterns the codebase already uses. Mention the things a stranger could not guess, such as why one module deliberately avoids a popular library, or which directory is generated and must never be edited by hand.

Context is not free, so be selective. Twenty relevant lines beat two thousand irrelevant ones. If the assistant keeps making the same wrong assumption, that is a signal the missing context is worth writing down permanently rather than repeating in every conversation.

Review the output as though a junior developer wrote it

Generated code is confident whether or not it is correct. It compiles, it reads well, and it may still be subtly wrong. Treat every suggestion as a draft from a capable but unfamiliar contributor.

- Check the edge cases first: empty inputs, missing files, network failures, and values at the boundaries.

- Verify that any external function or flag the assistant used actually exists in the version you have installed.
- Watch for silent behaviour changes, such as an error being swallowed instead of raised.
- Run the tests. If there are no tests, that absence is the first thing worth fixing.

Keep changes small enough to verify

An assistant will happily rewrite four hundred lines across six files in a single response. Accepting such a change is a bet that all of it is right, and reviewing it properly often takes longer than writing the change yourself would have. Smaller steps are easier to check, easier to revert, and easier to explain to whoever reads the commit history later.

Version control makes this practical. Commit before a large refactor. Ask for one coherent change at a time. When something goes wrong, you want the failure isolated to a diff you can still hold in your head.

Know where the tool is weakest

Assistants are strongest on well-trodden ground: common languages, popular frameworks, and problems that thousands of developers have already solved. They are weakest on anything recent, anything proprietary, and anything that depends on facts they cannot see. A model trained months ago does not know that a library changed its API last week, and it will not tell you that it is guessing.

They are also poor judges of your specific priorities. An assistant does not know that your team values readability over cleverness, or that a particular service has a strict latency budget, unless you say so. These are the judgements that remain yours.

Build a habit, not a dependency

Used well, an assistant removes drudgery and leaves more attention for design. Used carelessly, it produces a codebase nobody on the team fully understands. The difference is not the tool but the discipline around it: clear intent, adequate context, small verifiable steps, and honest review.

Developers who adopt that discipline report the same shift. They spend less time typing and more time deciding what should be built. That is a better use of a developer's day, and it is the real promise of working alongside an AI assistant.